

Agent Design Document

Personal Productivity and Task Execution Agent

1. Problem Statement

Knowledge workers and students routinely juggle scattered information across meeting notes, deadlines, and personal commitments, and struggle to convert that unstructured information into organized, actionable plans. Standard chatbots can answer questions about this information but cannot act on it — they cannot create tasks, schedule a day, or track progress over time. This project addresses that gap by building an AI agent that not only understands natural-language requests but reasons about intent, selects and executes real tools against a persistent database, and produces structured, actionable outputs (task lists, schedules, reports) rather than only conversational text.

The distinction this project is built around is explicit: a chatbot generates language in response to language; a workflow executes a fixed, predetermined sequence of steps regardless of input; an agent reasons about each individual request, decides dynamically whether action is required, chooses among multiple possible actions, and adapts its behavior, including pausing for human oversight, based on the nature of that action. The system described in this document is built to satisfy the agent definition specifically, not the simpler chatbot or fixed-workflow alternatives.

2. Users

The system is designed for a single individual managing their own workload — for example, a student balancing coursework deadlines with personal routines, or a professional managing client deliverables alongside meeting follow-ups. The current implementation is single-user per browser session (no authentication layer), making it suitable for personal productivity use or as a foundation for a future multi-tenant version.

A secondary, implicit user of the system is the human approver, who acts in a supervisory capacity when the agent proposes a sensitive action. In the current single-user design, the requester and approver are the same individual, but this mirrors how the approval mechanism would scale in a multi-user deployment, where a manager or team lead might review and approve actions requested by another user or performed by the agent on their behalf.

3. Use Cases

The agent supports the following major categories of user requests, each mapped to specific tools:

- Creating a single task with a title, due date, and priority (e.g. “Create a task to submit my report by Friday”)
- Extracting multiple tasks at once from unstructured meeting or class notes
- Listing and filtering tasks by status or priority
- Marking tasks complete or deleting them (both gated behind human approval)
- Generating a time-scheduled daily plan constrained by available hours, including deferred tasks and risk warnings

- Generating a 7-day weekly plan
- Producing a weekly productivity report with completion statistics
- Analyzing pending tasks to identify what is overdue or urgent, with a specific recommendation
- Summarizing notes into a structured breakdown of decisions and action items
- Saving and later searching free-text notes
- Drafting a follow-up email based on meeting notes

4. Agent Responsibilities

The agent is responsible for and permitted to:

- Interpret natural-language requests and determine whether a tool call is required
- Select the single most appropriate tool from its registry based on intent and tool descriptions
- Validate tool inputs against each tool's declared schema before execution
- Execute read operations (listing, searching, analyzing) and safe write operations (creating tasks/notes) without requiring approval
- Pause and request explicit human approval before executing actions classified as sensitive (completing or deleting a task)
- Generate natural-language responses summarizing tool results for the user
- Log every step of its decision process — intent analysis, tool selection, validation outcome, execution result, and approval status — to a persistent execution log

5. Agent Boundaries

The agent is explicitly restricted from:

- Executing any action outside its registered tool catalogue — it cannot invoke arbitrary code or access systems not exposed as a tool
- Completing or deleting a task without first obtaining human approval, regardless of how the request is phrased
- Answering questions using its own general training knowledge when a tool-based, data-grounded answer is expected (e.g. task or note queries are always answered from the database, not fabricated)
- Accessing or modifying data belonging to a different session/user — all task and note queries are scoped to the requesting session_key
- Sending real emails, modifying external systems, or taking any action with effects outside this application — draft_email produces a draft only, never a sent message
- Silently failing — any tool error, validation failure, or missing data is returned to the user as a clear message rather than an unexplained empty or incorrect result

6. Tool Catalogue

The agent has access to 13 registered tools, each with a defined input schema, description used for LLM-based selection, and an approval flag. All tools are registered centrally in tools/tool_registry.py and grouped by domain (tasks, planning, analysis, notes, email).

Tool	Approval	Inputs	Purpose
------	----------	--------	---------

create_task	No	title (req), description, due_date, priority	Creates a single new task
create_tasks_from_text	No	notes_text (req)	Extracts and creates multiple tasks from unstructured notes
list_tasks	No	status_filter, priority_filter	Lists tasks, optionally filtered
complete_task	Yes	task_id or task_title	Marks a task as completed
delete_task	Yes	task_id or task_title	Permanently deletes a task
generate_daily_plan	No	date, available_hours	Time-scheduled daily plan with deferred tasks and risk warnings
generate_weekly_plan	No	none	7-day task overview
generate_productivity_report	No	none	Weekly stats: completed, pending, overdue, high-priority counts
analyze_priorities	No	none	Identifies overdue/urgent tasks with a specific recommendation
summarize_notes	No	notes_text (req)	One-time summary into decisions and action items (not saved)
save_note	No	content (req), title	Permanently saves a note for later retrieval
search_notes	No	query (req)	Keyword search across saved notes
draft_email	No	notes_text (req), recipient_context	Drafts a follow-up email from notes

7. State Model

The system persists five categories of state in MySQL, all scoped by a session_key generated per browser session and stored client-side in localStorage:

- Task — title, description, due date, priority, status (pending/completed), creation timestamp
- Note — title, content, creation timestamp
- Plan — a snapshot of a generated daily/weekly plan or productivity report, including the task list at generation time, so historical plans remain accurate even if underlying tasks later change
- ApprovalRequest — the tool name, serialized input, and status (pending/approved/rejected) for any action requiring human sign-off
- ExecutionLog — a step-by-step record of every stage the controller passes through for a given request (intent_analysis, tool_selected, validation_failed, tool_executed, awaiting_approval, response_generated), each with a status and detail payload

Conversation history (AgentMessage) is also persisted so that chat state survives page navigation and refresh, rather than existing only in browser memory.

8. Approval Model

Each tool is registered with a requires_approval flag. Currently, complete_task and delete_task are the two tools flagged as sensitive, since both make an irreversible or semi-irreversible change to persistent data (marking a task permanently done, or removing it entirely). All other tools are read operations or additive writes (creating tasks/notes) that carry lower risk and do not block on approval.

When a flagged tool is selected, the controller does not execute it immediately. Instead, it creates an ApprovalRequest record with status “pending” and returns a message to the user indicating approval is required.

The user reviews the pending request — including the exact tool and input that would be executed — on a dedicated Approvals page, and can Approve or Reject it. Approval triggers execution of the original tool call; rejection marks the request as rejected and the tool is never invoked. Both outcomes are logged.

9. Error Strategy

The system handles four categories of failure distinctly, each logged with a status and surfaced to the user in plain language rather than a raw exception:

- Intent parsing failure — if the LLM's structured JSON output cannot be parsed, the system falls back to treating the request as a general query rather than crashing
- Unknown tool selection — if the LLM names a tool not present in the registry, the controller returns a clear “I don't have a tool called...” message instead of attempting execution
- Input validation failure — before execution, inputs are checked against the tool's schema; missing required fields or wrong types produce a specific error message listing what is missing, rather than allowing a malformed call to reach the database
- Tool execution exception — all tool calls are wrapped in a try/except block; unexpected errors (e.g. database issues) are caught, logged with status “failed”, and returned as a readable message rather than a server error page

A related but distinct case, ambiguous input (e.g. two tasks matching the same title fragment), is handled by returning the matching candidates and asking the user to specify an ID, rather than guessing or failing outright.

10. Security Considerations

- API keys and database credentials are stored in a .env file, excluded from version control via .gitignore, and loaded only through environment variables — never hardcoded in source
- All task, note, and message queries are filtered by session_key, preventing one session from reading or modifying another session's data
- Sensitive state-changing actions (complete_task, delete_task) require explicit human approval and cannot be triggered automatically by the agent's own reasoning alone
- SQL access is performed exclusively through SQLAlchemy's ORM query interface, which parameterizes queries and mitigates SQL injection risk rather than using raw string-concatenated SQL
- The LLM is only used for reasoning and text generation; it is never given direct database or filesystem access — all actual data mutations occur through the validated tool layer, keeping the LLM's influence bounded to selecting and parameterizing pre-defined, reviewed operations
- A documented limitation: the current implementation has no user authentication, so session isolation relies on an unguessable client-generated session key rather than a verified login — acceptable for a personal single-user demo, but noted as a required addition before any multi-user or production deployment.

11. Assumptions and Constraints

This design assumes a single active user per session, interacting through a web browser, with a continuously available MySQL instance and outbound network access to the OpenRouter API. It assumes tool execution is fast enough to complete within a single synchronous HTTP request cycle; no tool in the current catalogue performs long-running or asynchronous work. It also assumes the LLM's structured JSON output for intent analysis is usually well-formed; a parsing fallback exists for the cases where it is not, but no retry-with-correction loop is currently implemented.

A further constraint is that tool selection is limited to one tool call per user message. The controller does not currently support chaining multiple tool calls within a single reasoning pass (for example, automatically running

priority analysis and then generating a plan in response to one request). This was a deliberate scope decision to keep the control flow linear, transparent, and easy to debug during development, at the cost of not supporting more complex multi-step workflows without an additional user message per step.

12. Future Enhancements

- Multi-step tool chaining within a single request, using a graph-based orchestration approach (e.g. LangGraph) once workflows require conditional branching across multiple tool calls
- User authentication, replacing the client-generated session key with a verified login and per-user data isolation enforced at the database layer
- Persisting approval and rename actions with full audit trails, including who approved an action and when, for accountability in a multi-user setting
- Automated regression testing via a formal test framework (e.g. Pytest) layered on top of the existing evaluation test cases
- Structured input validation using a schema library (e.g. Pydantic) to replace the current hand-written validator with stronger type guarantees and clearer error messages